Talking Points: Mid-Semester Research Milestone

Opol Charis | S24B23/003 | Investigate, Recommend or Escalate: A Rule-Constrained Reinforcement Learning Agent for IT Incident Triage under Ambiguous Evidence

1. The five minutes at a glance

You have about five minutes: two to three to talk, about two for questions. The panelist is checking four things: the research direction, the technical approach, what is done, and whether a prototype and poster are realistic in six weeks.

Time	Part of the page	The one thing to land
0:00–0:30	Problem and objective	Staff must act before they know the cause; current AI cannot ask for more evidence or say it is unsure.
0:30–1:05	What exists and the gap	Three ideas exist, but only two at a time. Nobody has put all three choices in one agent for IT incidents.
1:05–1:55	Pipeline diagram	Point at the two orange boxes. The simulator and the agent with its rule checker are the new work.
1:55–2:20	Progress to date	Review, three chapters and the full design are done. No code yet, and you say so first.
2:20–2:45	Next six weeks	One activity per week, ending in a working prototype and a fair comparison.
2:45–5:00	Questions	Short, honest answers. Section 5 has them ready.

Delivery. Hand over the page, then talk to the panelist, not to the page. Use your finger on the diagram when you reach it. If you are stopped early, say the 30-second version in Section 3.

2. The script (about two and a half minutes)

Read this aloud twice tonight with a timer. Do not memorise it word for word; memorise the order.

[Problem, 30 s]

"When an IT incident is reported, for example the student portal is slow, the technician has to act before knowing the cause. It could be the database, the network or a full disk, and the first evidence is often incomplete or contradictory. The technician has three choices: check further, try a likely fix, or pass it to someone more senior. Today's AI tools for this give one answer per ticket. They cannot ask for more evidence, and they cannot say 'I am not sure.' My objective is to build and test an AI agent that learns when to check further, when to recommend a fix that the rules allow, and when to hand over to a human."

[What exists and the gap, 35 s]

"Three lines of research matter here. First, AI triage tools classify the ticket from the report alone, but a two-year study at Microsoft found that symptoms often do not reveal the cause, so tickets bounce between teams. Second, a technique called shielding stops a learning agent from breaking rules, but it has been tested mainly on robots and games. Third, learning to defer lets a model pass hard cases to a person, but without the option to gather more evidence first. In the 46 sources I reviewed, these ideas only appear two at a time. Nobody has put all three choices in one agent for IT incidents, checked it against rules, and tested it as the evidence gets worse. That is my gap."

[Pipeline, 50 s: point at each box]

"Here is the pipeline. The input is how incidents are really handled, which I am collecting from our own Directorate of University ICT Services: causes, symptoms, checks and operating rules. From that I build an incident simulator, which is new. It has 25 causes, 20 checks and 10 rules. Each simulated incident has a hidden cause, and I can make the evidence missing, wrong or confusing on purpose. On the right is the proposed agent. At each step it picks one of three things: run a check, recommend a fix, or hand over. Every choice goes through a separate rule checker. If the choice is allowed, it goes ahead. If it is blocked, the agent gets one more try. If it is blocked again, the incident goes to a technician. On the left are six simpler strategies that I compare against, on the same incidents. At the bottom are the four things I measure: correct and wrong fixes, the cost of checking, whether the right incidents are handed over, and rule breaks."

[Progress, 25 s]

"So far, the literature review is done, Chapters One to Three are written, which is a 40-page draft I have here, and the full design is specified. The Directorate has agreed to support the study. I want to be clear about what is not done: there is no code yet, and no experiments, so I am claiming no results today."

[Next six weeks, 25 s]

"In week one I collect the scenarios and build the simulator. Week two, the rule checker and the comparison strategies. Week three, I train the agent. Weeks four and five are the experiments, including making the evidence worse. Week six is the prototype and poster. The prototype is the simulator and the trained agent, with a simple screen that replays an incident step by step next to the comparison results."

3. The 30-second version

"IT technicians often have to act on an incident before they know the cause. Current AI tools give one answer and cannot ask for more evidence or admit they are unsure. I am building an AI agent that chooses between checking further, recommending a fix, and handing over to a human, with a separate rule checker that blocks actions the organisation does not allow. I test it in a simulator built with our ICT Directorate, against six simpler strategies, and I measure accuracy, cost, hand-overs and rule breaks as the evidence gets worse. The review and design are done; the build starts this week."

4. Going deeper, section by section

4.1 The problem

- **Triage** means the early decision about what happens to an incident next. It happens before the cause is known.
- **Ambiguity** means the evidence fits more than one cause. It has three sources, and each is a dial in the simulator: evidence that is *missing*, evidence that is *wrong*, and evidence that *looks the same for several causes*.
- Every choice has a cost. Checking takes time. A wrong fix can make things worse. Handing over uses a senior person's time.
- The portal example is an illustration you made up to explain the idea. Do not present it as a real Directorate incident.

4.2 The three references

Source	What they did and found	Limit that matters to you
[1] Chen et al., 2020 (ESEC/FSE)	Studied two years of incident management at Microsoft. Symptoms are often not enough to find the responsible team, because different problems look alike, so incidents are reassigned several times.	Their AI component predicts a team from the ticket text. It does not decide to look for more evidence or to hand over.
[2] Alshiekh et al., 2018 (AAAI)	Introduced the "shield": a separate component that watches each action of a learning agent and overrides it only when it would break a stated rule.	Tested on grid-world robots, a self-driving scenario, an Atari game and a water tank. Not on IT operations, and it needs a formal model of the environment.
[3] Madras et al., 2018 (NeurIPS)	Introduced "learning to defer": a model learns when to pass a case to a human, taking that person's ability into account.	A one-step choice between deciding and deferring. There is no option to gather more evidence first.

Two more to keep in your pocket (not on the page, but good if asked):

- **Janisch et al., 2019 (AAAI):** an agent learns which pieces of costly information to acquire before it classifies. This is the "check further" idea. It has no hand-over option.
- **Alauthman et al., 2026:** a learning agent with a safety shield for incident response in industrial control systems. This is the closest work to yours. Their agent picks responses; it does not choose whether to investigate or hand over.

Careful. You have read Alauthman et al. only at abstract level. If you mention it, say "from their abstract". Read the full paper before the final defence.

4.3 The gap, in full

The one-page summary gives the short version. Chapter Two states it in five parts:

1. **No test bed.** There is no simulator for IT service incidents where ambiguity can be set on purpose. Existing ones model cyberattacks and microservice faults.
2. **No three-way decision.** Some studies offer "check further or decide". Others offer "decide or hand over". None trains one agent on all three.
3. **Rules and the decision.** Nobody reports how a rule checker behaves when the agent can also investigate or hand over.
4. **No joint measurement.** Accuracy, checking cost, hand-overs and rule compliance have each been measured, but never together for one agent.
5. **No robustness picture.** Nobody shows how the mix of the three choices shifts as evidence gets worse.

4.4 Why this is research and not just an implementation

The guideline says applying an existing model to existing data is not enough. Your answer has three parts:

- **You build the test bed.** There is no existing dataset or environment for this decision, so the simulator is itself a contribution.
- **You investigate a question, not a product.** Can one agent learn to treat "check further" and "hand over" as different responses to doubt, or does it collapse into one of them?
- **You compare fairly.** Six strategies, the same incidents, four measures, several levels of ambiguity. One of the six, the confidence-threshold strategy, is deliberately strong, and you do not assume your agent will beat it.

4.5 The pipeline, box by box

Input. Knowledge from the Directorate of University ICT Services: what causes incidents, how they are first reported, what technicians check, and what rules govern their actions. No personal data.

Incident simulator (new).

- 25 causes across four categories: database failures, network degradation, authentication failures, storage exhaustion.
- 20 diagnostic checks, each with a cost. 10 operating rules.
- Each incident has one hidden cause. Each check has a probability of coming back positive for each cause. Those probabilities are the heart of the simulator.
- The agent has **46 possible actions**: 20 checks, 25 fixes (one per cause) and 1 hand-over.
- Three dials for ambiguity: the chance a check is unavailable, the chance a result is wrong, and how alike the causes in a category look.

Proposed agent (new).

- A reinforcement learning agent: it learns by trial and error from rewards, with no labelled examples.
- The algorithm is **DQN** (deep Q-network). It suits a small set of distinct actions. PPO is a second algorithm you will try only if time allows.
- Rewards: a correct fix is rewarded, a wrong fix is penalised heavily, each check costs a little, and a hand-over costs a moderate amount. So: correct fix is better than hand-over, which is better than a wrong fix.

Rule checker (new).

- A separate component. The rules are written down as plain conditions that a person can read, not learned.
- **Allowed** $\rightarrow$ the action goes ahead. **Blocked** $\rightarrow$ the agent is told and tries once more. **Blocked again** $\rightarrow$ the incident is handed to a technician.
- Handing over is always allowed, so there is always a legal move.
- Example rule, for illustration only: "do not recommend restarting a production database without approval". The real ten rules come from the Directorate.

Six comparison strategies.

Strategy	What it does	What it stands for
Guess from first symptoms	Picks the most likely cause straight away	Today's one-answer-per-ticket tools
Fixed checklist	Runs a set list of checks, then picks	Runbook practice
Always hand over	Hands every incident to a human	The "safe" extreme
Act when confident enough	Keeps checking until one cause is likely enough, otherwise hands over	A strong hand-built strategy
Agent with no rule checker	The same agent without the checker	Shows what the checker adds
Agent with penalties only	Rule breaks are punished but not blocked	Tests whether punishment alone is enough

Four measures.

- **Decision quality:** how many incidents the agent fixes correctly, and how many fixes are wrong.
- **Cost of checking:** how many checks it runs and what they cost.
- **Hand-overs:** how many it hands over, and whether those were the right ones to hand over.
- **Rule compliance:** how often it tries a forbidden action, and how often one actually goes through.

All four are measured again at each level of ambiguity. Each experiment is repeated with 10 different random seeds, and the results are averaged.

4.6 Progress: what you can show

- **Literature review.** Five databases searched; 46 sources reviewed; a table comparing the 11 closest studies. Show Chapter Two and the search exports.
- **Report.** Chapters One to Three, compiled as a 40-page draft. Bring it printed or on your laptop.
- **Design.** Chapter Three specifies the simulator, the agent, the rule checker, the comparisons, the measures and four experiments.
- **Partner.** The Directorate has agreed to support the study.

Careful. Say "no code yet" before you are asked. The page says it, and a panelist will respect it more from you than from their own question.

4.7 The six weeks, with the risks

Week	Work	What you have at the end
1	Collect causes, checks and rules; build the simulator	A simulator that generates incidents
2	Rule checker; six comparison strategies; staff review	Simple strategies running; scenarios checked by staff
3	Train the agent	A trained agent and learning curves
4	Compare with the six strategies	The main results table
5	Worse evidence; analysis	Robustness charts
6	Prototype, poster, results	Demo screen and poster

If time runs short, cut in this order: the second algorithm (PPO), then the second way of training, then the number of random seeds from ten to five. The simulator, the DQN agent, the rule checker, the six comparisons and the four measures stay.

Biggest risk: week one depends on the Directorate. If their input is late, start the simulator with your own draft scenarios and replace them when theirs arrive. This fallback is a suggestion, not yet agreed with your supervisor.

5. Likely questions and short answers

Q: What is actually new here? Shielding and reinforcement learning both exist.

Yes, and I do not claim either is new. A shielded agent has even been proposed for incident response in industrial control systems. What is new is the decision I study: one agent choosing among checking further, recommending and handing over, for IT service incidents, under rules, and measured as evidence gets worse. No study I reviewed does that.

Q: Why reinforcement learning? Why not a classifier or plain rules?

A classifier gives one answer per ticket. It cannot decide to run a check first. Plain rules cannot weigh whether

another check is worth its cost for this particular incident. Triage is a sequence of decisions with costs, and that is what reinforcement learning is designed for. Both alternatives are among my six comparisons, so the data will show whether I am right.

Q: Why a simulator and not real data?

Three reasons. Real incident data is confidential and hard to get; even the Microsoft study reported only relative figures. A learning agent needs to try actions and see outcomes, which cannot be done on live systems. And I need to control ambiguity, which only a simulator allows. The cost is that results hold for the simulator, and I say so.

Q: How do you know the simulator is realistic?

Its content comes from the Directorate's practice, and Directorate staff review generated incidents before I fix it. It is still a simplification: one cause per incident, one fix per cause. I state those assumptions.

Q: DQN or PPO?

DQN is the main algorithm. It fits a small set of distinct actions and is what the closest studies use. PPO is a check I will run only if time allows, to see whether the findings depend on the algorithm.

Q: What if the agent learns to hand over everything?

That is why I do not measure accuracy alone. "Always hand over" is one of my comparison strategies. It breaks no rules and makes no wrong fix, but it resolves nothing itself. Handing over also has a cost in the reward, so the agent is pushed away from it.

Q: How do you decide which incidents *should* be handed over?

The simulator knows the true cause, so it can label each incident. An incident should be handed over if the rules forbid its fix, or if even with every available check the most likely cause is still the wrong one.

Q: What happens when the rule checker blocks an action?

The agent is told, and it chooses again once. If the second choice is also blocked, the incident goes to a technician. I count those forced hand-overs separately from the ones the agent chose.

Q: If the checker blocks everything forbidden, is "fewer violations" not guaranteed?

Yes, so that is not my claim. I measure how often the agent *tries* forbidden actions, whether that falls as it learns, and whether it still fixes as many incidents correctly within the rules as the agent without a checker.

Q: What does the Directorate provide? What about confidentiality?

Their knowledge of incident causes, checks and rules, and a review of my scenarios. No personal data, credentials or system details go into the simulator or the report. We are finalising a confidentiality agreement, and the agent never touches a live system.

Q: Is your review a systematic literature review?

It is a structured thematic review, organised by my five research questions. I searched five databases and screened the results by keyword. I do not call it a completed systematic review, and Chapter Two says so.

Q: Why 25 causes, 20 checks and 10 rules?

Large enough that the task is not trivial: several causes in each category share symptoms. Small enough that people at the Directorate can review every scenario and rule.

Q: Why not use a large language model?

Language models are good at reading tickets, and the literature uses them for that. My question is about the decision that follows: check, fix or hand over, under rules. A small learning agent in a simulator lets me measure that decision exactly and repeat it. A language model could later supply the evidence that my agent reasons over.

Q: Can you really finish in six weeks?

The design is fixed, the tools are standard (Python, Gymnasium, Stable-Baselines3), and the simulator is small. I have a cut list if time is short: the second algorithm goes first. The core stays.

Q: What will the prototype look like?

A program that generates an incident, shows the agent's steps one by one (each check and its result, each proposal, any block, and the final decision) and shows the comparison results next to it.

Q: Would this work in a real IT department?

I do not claim that. This study shows how the method behaves under controlled conditions. In the design the agent only recommends; it never executes anything. A real deployment would need real evaluation with technicians.

Q: What is the weakest part of your study?

The simulated technician always resolves a handed-over incident. Real reviewers get overloaded, so my study understates the cost of heavy hand-over. I report the hand-over rate so that cost stays visible.

6. Numbers to have ready

46	Sources reviewed in Chapter Two; also the number of actions the agent has (a coincidence)
5	Databases searched: Scopus, IEEE Xplore, ACM Digital Library, SpringerLink, SIAM
11	Closest studies compared in Table 2.1
25 / 20 / 10	Causes / checks / rules in the simulator
4	Incident categories; also the number of measures and of experiments
20 + 25 + 1	Check actions + fix actions + hand-over
3	Choices the agent has; dials for ambiguity; references on the page
6	Comparison strategies; weeks remaining
10	Random seeds per experiment
40 pages	Compiled draft of Chapters One to Three
2 years	Length of the Microsoft study in reference [1]

7. Plain words for technical terms

If they say	It means
Reinforcement learning (RL)	Learning by trial and error from rewards, without labelled examples
Agent / policy	The decision-maker / its learned way of choosing actions
DQN	An RL algorithm that learns how good each action is in each situation
Environment	The simulator the agent practises in
Episode	One incident, from report to final decision
Partial observability	The agent cannot see the true cause, only evidence about it
Shield / safety mechanism	Your rule checker
Escalation / deferral	Handing the incident to a human
Baseline	A simpler strategy used for comparison
Ablation	Removing one part to see what it contributed
Robustness	How well performance holds up as conditions get worse
AIOps	Using AI to help run IT operations

8. What not to say

- **Do not claim results.** No experiment has run. Say “I expect” or “I will test whether”, never “the agent achieves”.
- **Do not say “no one has done this”.** Say “none of the studies I reviewed does this”.
- **Do not say the system is “safe”.** The checker enforces the ten written rules inside a simulator. That is all it guarantees.
- **Do not say it is deployed, or will be.** The agent recommends inside a simulation and never touches a live system.
- **Do not say you have Directorate data.** You have their agreement. Collection starts in week one.
- **Do not overstate the reading.** Eight of the 46 sources were assessed from their abstracts. If asked about one in detail, say so.

9. What to bring

- The printed one-page summary, single-sided A4. Bring a spare.
- The 40-page draft (`main.pdf`), printed or open on your laptop.

- The search exports folder and the notes files, on your laptop, as evidence of the review.
- These notes.